

CLASE 06 — TU PROYECTO ASIGNADO

Primera implementación física en PostgreSQL: DataGrip, DDL y constraints

Guía práctica del estudiante · Observa ParkFlow 360 y transfiere el método a tu proyecto

REGLA CENTRAL: ParkFlow 360 es el proyecto del profesor. NO debes copiar sus tablas ni sus nombres. Debes observar cómo el profesor transforma su diseño en PostgreSQL y repetir el mismo MÉTODO sobre el proyecto asignado a tu grupo.

1. Qué cambia en esta clase

En las clases anteriores tu equipo diseñó el dominio, el modelo conceptual, el modelo relacional, el DER lógico, el diccionario de datos y el diseño físico preliminar. Hoy ese trabajo comienza a existir de verdad dentro de PostgreSQL. Por primera vez crearás físicamente una parte de la base de datos de TU proyecto.

Hasta hoy: diseño. Desde hoy: diseño + ejecución + verificación + evidencia reproducible.

2. Qué debes aprender hoy

- Entender la diferencia entre PostgreSQL y DataGrip.
- Conectarte a PostgreSQL desde DataGrip.
- Crear una base de datos específica para tu proyecto.
- Traducir 4–6 tablas núcleo de tu diseño físico a CREATE TABLE.
- Implementar PK, FK, NOT NULL, UNIQUE y CHECK donde correspondan.
- Respetar el orden de creación por dependencias.
- Probar al menos tres violaciones de integridad de forma controlada.
- Guardar el DDL como migración versionada en Git.
- Explicar cada constraint que hayas escrito.

3. Qué debes observar mientras el profesor trabaja ParkFlow 360

Lo que hará el profesor	Lo que debes aprender, NO copiar
Crearé la base parkflow360	Cómo nombrar y crear una base de proyecto.
Crearé parking_lot antes de zone	Cómo ordenar tablas según dependencias FK.
Usaré UNIQUE compuesto en zone	Cómo expresar una unicidad que depende del contexto.
Usaré CHECK para estados/tipos	Cómo impedir valores determinísticamente inválidos.
Provocaré errores de integridad	Cómo verificar que PostgreSQL realmente protege los datos.

Guardaré V1_init_core.sql	Cómo convertir el esquema local en evidencia reproducible.
---------------------------	--

4. Lo que NO debes copiar de ParkFlow

- No copies parking_lot, zone, parking_space, customer o vehicle si esas entidades no existen en tu proyecto.
- No copies los estados AVAILABLE/OCCUPIED/BLOCKED.
- No copies nombres de constraints sólo cambiando una palabra.
- No uses la misma cardinalidad si tu ficha oficial define otra.
- No copies tipos de datos sin revisar los atributos de tu dominio.
- No inventes tablas nuevas que contradigan la ficha oficial del proyecto.

5. Archivos que debes tener abiertos antes de empezar

- ☐ Ficha oficial de tu proyecto asignado.
- ☐ Modelo conceptual de Clase 02.
- ☐ Modelo relacional de Clase 03.
- ☐ DER lógico + diccionario de Clase 04.
- ☐ Modelo físico y plan de migración de Clase 05.
- ☐ Repositorio Git actualizado.

6. Paso 1 — Identifica tus primeras 4–6 tablas núcleo

No intentes construir todo el proyecto en una sola clase. Selecciona el núcleo mínimo que permita demostrar relaciones reales y restricciones útiles.

Pregunta	Tu respuesta
¿Cuál es la entidad raíz o catálogo principal?	_____
¿Qué tabla depende directamente de ella?	_____
¿Qué segunda relación 1:N puedes implementar?	_____
¿Qué entidad de actor o recurso necesita persistencia?	_____
¿Qué tabla transaccional conviene dejar para después?	_____

Ejemplo de transferencia: BiblioSmart podría comenzar por book → copy y reader; EquipRent por equipment → unit y customer; CargoTrack por contact → shipment → package. Son ejemplos de patrón, no soluciones obligatorias.

7. Paso 2 — Crea la base de datos de TU proyecto

El nombre debe ser técnico, corto y consistente. Ejemplos posibles: bibliosmart, cargotrack, salonpro. Usa el nombre acordado por tu equipo.

```
CREATE DATABASE nombre_de_tu_proyecto
WITH
OWNER = postgres
ENCODING = 'UTF8';
```

Sustituye nombre_de_tu_proyecto. No copies literalmente ese nombre.

8. Paso 3 — Crea la carpeta de migraciones

```
backend/src/main/resources/db/migration/
```

Archivo de esta clase:

```
backend/src/main/resources/db/migration/V1__init_core.sql
```

9. Paso 4 — Escribe primero el DDL en archivo, no directamente “a mano” en la base

Tu archivo V1__init_core.sql es la fuente reproducible. Si mañana borras la base, deberías poder reconstruir este núcleo ejecutando el script.

10. Plantilla genérica de CREATE TABLE

Esta plantilla es una estructura didáctica. Debes reemplazar nombres, columnas, tamaños, constraints y relaciones según TU modelo.

```
CREATE TABLE nombre_tabla (
  id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  campo_obligatorio VARCHAR(100) NOT NULL,
  campo_opcional VARCHAR(150),
  activo BOOLEAN NOT NULL DEFAULT TRUE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT uq_nombre_tabla_campo UNIQUE (campo_obligatorio),
  CONSTRAINT ck_nombre_tabla_campo CHECK (BTRIM(campo_obligatorio) <> '')
);
```

11. Plantilla genérica para una tabla hija con FK

```
CREATE TABLE tabla_hija (
  id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  tabla_padre_id BIGINT NOT NULL,
```

```

codigo VARCHAR(30) NOT NULL,
created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT fk_tabla_hija_padre
    FOREIGN KEY (tabla_padre_id) REFERENCES tabla_padre(id),
CONSTRAINT uq_tabla_hija_padre_codigo
    UNIQUE (tabla_padre_id, codigo)
);

```

No uses tabla_padre/tabla_hija en tu entrega final. Deben ser nombres reales de tu dominio.

12. Paso 5 — Verifica cada decisión antes de ejecutar

Decisión	Pregunta de control
PRIMARY KEY	¿Esta columna identifica inequívocamente la fila?
FOREIGN KEY	¿La tabla hija realmente depende de ese padre?
NOT NULL	¿El registro puede existir válidamente sin ese dato?
UNIQUE	¿El negocio prohíbe duplicarlo? ¿Global o dentro de un padre?
CHECK	¿La regla puede expresarse de forma determinista en una fila?
DEFAULT	¿El valor por defecto representa realmente el estado inicial?

13. Paso 6 — Ejecuta en orden de dependencias

1. Ejecuta primero una tabla sin FK.
2. Refresca DataGrip y revisa columnas/constraints.
3. Ejecuta la primera tabla hija.
4. Comprueba visualmente la FK.
5. Continúa hasta completar únicamente el núcleo seleccionado.
6. No crees todavía tablas que tu modelo aún no puede justificar.

14. Paso 7 — Inserta un dataset mínimo válido

Escribe al menos una fila válida por cada tabla núcleo. Debes hacerlo con INSERT y conservar esos comandos en un archivo de laboratorio o sección separada de tu documentación.

```

INSERT INTO nombre_tabla (campo_obligatorio)
VALUES ('dato de prueba');

SELECT * FROM nombre_tabla;

```

15. Paso 8 — Oblígate a comprobar que las restricciones funcionan

Prueba mínima	Qué debes intentar
NOT NULL	Omitir un dato obligatorio.
UNIQUE	Insertar un duplicado que el negocio prohíbe.
FOREIGN KEY	Referenciar un ID padre inexistente.
CHECK	Enviar un estado, cantidad o valor fuera del dominio permitido.

No corrijas el error inmediatamente. Lee el mensaje de PostgreSQL, identifica el constraint y explica por qué se activó. Esa explicación forma parte del aprendizaje.

16. Paso 9 — Inspecciona la estructura mediante SQL

```
SELECT table_name
FROM information_schema.tables
WHERE table_schema = 'public'
ORDER BY table_name;

SELECT table_name, column_name, data_type, is_nullable
FROM information_schema.columns
WHERE table_schema = 'public'
ORDER BY table_name, ordinal_position;
```

17. Paso 10 — Registra evidencia en tu documentación

Actualiza o crea una sección de evidencia de Clase 06 con la siguiente información:

- Nombre de la base creada.
- Tablas incluidas en V1 y por qué fueron seleccionadas.
- Orden de creación.
- Listado de PK y FK.
- UNIQUE y CHECK implementados.
- Captura o descripción de al menos tres errores de integridad probados.
- Decisiones pendientes para la siguiente clase.

18. Paso 11 — Versiona con Git

```
git status
git add backend/src/main/resources/db/migration/V1__init_core.sql
git commit -m "feat(db): create initial core schema"
```

El mensaje puede adaptarse, pero debe describir el cambio real. No uses mensajes como “avance”, “base”, “cosas” o “final”.

19. Revisión cruzada obligatoria dentro del equipo

Integrante que revisa	Qué debe comprobar
Revisor A	Las tablas coinciden con el DER de Clase 04.
Revisor B	Todas las FK tienen sentido y apuntan a PK existentes.
Revisor C	NOT NULL/UNIQUE/CHECK están justificados por reglas reales.
Revisor D (si existe)	El script puede ejecutarse desde cero sin depender de cambios manuales.
Revisor E (si existe)	Los nombres y convenciones son consistentes.

20. Errores que pueden bajar la calidad de tu trabajo

- Crear tablas que no aparecen en el análisis del proyecto.
- Copiar las tablas de ParkFlow 360.
- Usar VARCHAR para absolutamente todo.
- Omitir FK para evitar errores.
- Crear tablas manualmente desde la interfaz y no guardar SQL.
- No probar restricciones.
- Usar nombres como tabla1, datos, detalle, maestro sin significado de dominio.
- Escribir una V1 que sólo funciona porque antes hiciste cambios manuales en DataGrip.
- Guardar usuario/contraseña de PostgreSQL dentro del repositorio.

21. Preguntas que debes poder responder al terminar

- ¿Qué diferencia existe entre PostgreSQL y DataGrip?
- ¿Cuál es la tabla padre principal de tu V1 y por qué?
- ¿Qué FK implementaste y qué relación representa?
- ¿Qué dato tiene UNIQUE y cuál es la regla de negocio que lo justifica?
- ¿Qué CHECK implementaste?
- ¿Qué error obtuviste cuando violaste una FK?
- ¿Por qué tu script puede considerarse reproducible?
- ¿Qué tablas decidiste NO crear todavía y por qué?

22. Evidencia mínima obligatoria de la Clase 06

☐ DataGrip conectado a PostgreSQL.

- ☐ Base de datos del proyecto creada.
- ☐ Entre 4 y 6 tablas núcleo creadas físicamente.
- ☐ PK/FK verificables.
- ☐ Al menos un UNIQUE y un CHECK donde el dominio lo justifique.
- ☐ Dataset mínimo insertado.
- ☐ Al menos tres errores de integridad provocados y comprendidos.
- ☐ V1_init_core.sql dentro del repositorio.
- ☐ Commit descriptivo.
- ☐ Todos los integrantes pueden explicar las decisiones principales.

23. Autoevaluación rápida

Nivel	Criterio
Verde	Puedo borrar la base, recrearla con mi script y explicar cada constraint.
Amarillo	El script funciona, pero todavía no puedo justificar algunas decisiones.
Rojo	Mi base depende de cambios manuales, no sé qué hacen mis FK o copié estructuras sin comprenderlas.

24. Definition of Done — Tu equipo

- ☐ El DDL coincide con el diseño previamente aprobado por el equipo.
- ☐ El script ejecuta desde una base limpia.
- ☐ Las tablas se crearon en orden correcto.
- ☐ Las restricciones básicas fueron verificadas con casos positivos y negativos.
- ☐ No existen secretos versionados.
- ☐ La evidencia está documentada.
- ☐ El cambio está en Git.
- ☐ El equipo puede defenderlo sin depender del integrante que escribió el SQL.

25. Preparación para la Clase 07

La siguiente clase utilizará la base que hoy construiste. Prepárate para trabajar DML y consultas: INSERT, UPDATE, DELETE controlado, SELECT, WHERE, ORDER BY, operadores, tratamiento de NULL y construcción de datos de prueba reproducibles. Si tu V1 no puede recrearse limpiamente, tendrás problemas en la siguiente clase: corrígelo antes de avanzar.

Principio del curso: no avances construyendo encima de una base que no puedes explicar ni reproducir.